

Surprise Quiz–5 : Solutions

1. Consider two decision problems Q_1, Q_2 such that Q_1 reduces in polynomial time to 3-SAT and 3-SAT reduces in polynomial time to Q_2 . Then which one of the following must be true?_____

- (A) Q_2 is **NP-hard**
- (B) Q_1 is **NP-hard**
- (C) Both Q_1 and Q_2 are in **NP**
- (D) Both Q_1 and Q_2 are **NP-hard**

Solution: A

Explanation:

Recall the core properties of a polynomial-time reduction ($A \leq_p B$):

- If $B \in P$, then $A \in P$.
- If A is NP-hard, then B is NP-hard.

We are given the following reductions:

- (a) $Q_1 \leq_p 3\text{-SAT}$
- (b) $3\text{-SAT} \leq_p Q_2$

We know that 3-SAT is an **NP-complete** problem, meaning it is both in **NP** and **NP-hard**.

Analysis of the problems:

- **Status of Q_2 :** Since $3\text{-SAT} \leq_p Q_2$ and 3-SAT is NP-hard, Q_2 must be at least as hard as 3-SAT. Therefore, Q_2 is **NP-hard**.
- **Status of Q_1 :** Since $Q_1 \leq_p 3\text{-SAT}$ and 3-SAT is in NP, Q_1 must be at least as easy as 3-SAT. Therefore, $Q_1 \in \text{NP}$. We cannot guarantee it is NP-hard.

Evaluating the options:

- (A) **True.** The reduction from 3-SAT guarantees Q_2 is NP-hard.
- (B) **False.** Q_1 is only guaranteed to be in NP; it could be in P.
- (C) **False.** Q_2 is NP-hard, but not necessarily in NP (it could be outside NP).
- (D) **False.** As established, Q_1 is not guaranteed to be NP-hard.

Correct answer: (A)

2. In the 3-SAT $\leq$ IND-SET reduction, $G(\varphi)$ is built by creating a triangle per clause (one vertex per literal) and adding conflict edges between x_i and $\neg x_i$ across triangles.

If φ has 7 clauses, what is the correct target independent-set size $k(\varphi)$?_____

Solution: 7

Explanation:

$$k(\varphi) = 7.$$

Each triangle contributes exactly one vertex to any maximum independent set, since no two vertices inside the same triangle can both be selected (they are mutually adjacent). Hence an independent set of size $k(\varphi)$ must pick exactly one vertex from each of the 7 triangles.

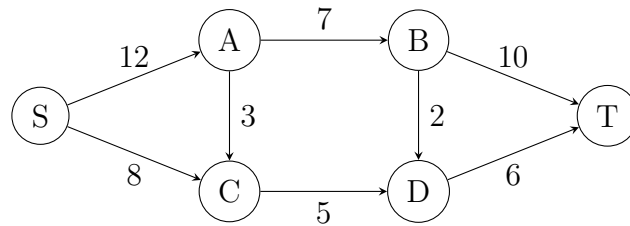
Why is this the right target size? The reduction satisfies:

$$\varphi \text{ is satisfiable} \iff G(\varphi) \text{ has an independent set of size } k(\varphi).$$

- ($\Rightarrow$) A satisfying assignment sets at least one literal per clause to **true**; picking the corresponding vertex from each triangle yields an independent set of size $k(\varphi)$ (conflict edges are never violated since we cannot simultaneously set x_i and $\neg x_i$ to **true**).
- ($\Leftarrow$) An independent set of size $k(\varphi)$ contains one vertex per triangle; setting the corresponding literals to **true** gives a consistent, satisfying assignment.

For φ with 7 clauses: $\boxed{k(\varphi) = 7}$.

3. Consider the following directed flow network (all edges are unidirectional and labeled with capacities. Residual edges are not present initially and must be inferred after augmentation.):



Consider the path:

$$S \rightarrow A \rightarrow B \rightarrow D \rightarrow T$$

Answer the following:

- Bottleneck capacity of the path _____
- Residual capacity of reverse edge $B \rightarrow A$ after augmentation _____

Solution: Bottleneck: 2, Residual capacity of Reverse Edge: 2

Explanation:

Given Path:

$$S \rightarrow A \rightarrow B \rightarrow D \rightarrow T$$

Step 1: Bottleneck Capacity

The bottleneck capacity of an augmenting path is the minimum capacity among all edges in the path.

$$S \rightarrow A = 12$$

$$A \rightarrow B = 7$$

$$B \rightarrow D = 2$$

$$D \rightarrow T = 6$$

$$\text{Bottleneck} = \min(12, 7, 2, 6) = 2$$

Step 2: Residual Capacity of Reverse Edge $B \rightarrow A$

Residual capacity of reverse edge (B to A) = current flow on original forward edge (A to B) = maximum flow you can cancel(2).

Initially, there is no reverse edge, so its capacity is 0.

When a flow of 2 units is sent along the path, the residual graph is updated as follows:

- Forward edge capacity decreases by 2
- Reverse edge capacity increases by 2

Thus, Residual capacity of

$$B \rightarrow A = 2$$

Final Answer: Bottleneck = 2, Residual capacity of Reverse Edge = 2

4. The Miso Good food truck has produced $m = 3$ types of food items $\{b_1, b_2, b_3\}$ with quantities q_j . There are $n = 4$ customers $\{a_1, a_2, a_3, a_4\}$, each providing a set of acceptable choices A_i .

Inventory and Preferences

The quantities available for each food item are:

- q_1 (Miso Ramen) = 2
- q_2 (Spicy Tofu) = 1
- q_3 (Sushi Roll) = 1

The customer preferences are:

- $A_1 = \{b_1, b_2\}$
- $A_2 = \{b_1\}$
- $A_3 = \{b_1\}$
- $A_4 = \{b_2, b_3\}$

Using a maximum flow algorithm, find the maximum number of customers that can be satisfied.

Solution: 4

Explanation:

Step 1: Problem Summary Food items (Supply):

$$b_1(2), \quad b_2(1), \quad b_3(1)$$

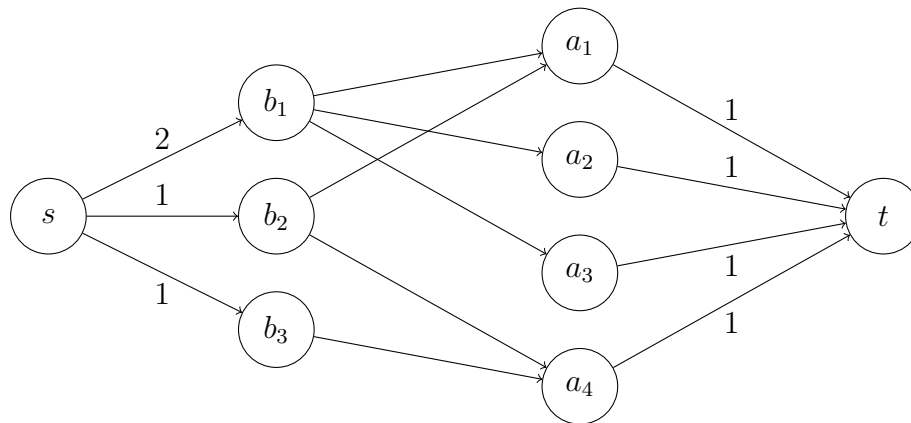
Customers and preferences (Demand):

- $A_1 = \{b_1, b_2\}$
- $A_2 = \{b_1\}$
- $A_3 = \{b_1\}$
- $A_4 = \{b_2, b_3\}$

Step 2: Flow Network Construction (Food to Customers) We construct a bipartite flow network where the flow represents the movement of food to the customers:

- **Source s :** Connects to each food item b_j with capacity equal to its quantity.
- **Bipartite Edges:** Food items b_j connect to customers a_i who prefer them (capacity 1).
- **Sink t :** Each customer a_i connects to sink t with capacity 1.

Step 3: Flow Network Diagram



Step 4: Flow Assignment A feasible maximum flow of value 4 is achieved with the following distribution:

- **Food b_1 :** 1 unit to a_2 and 1 unit to a_3 (Total 2/2 used).
- **Food b_2 :** 1 unit to a_1 (Total 1/1 used).
- **Food b_3 :** 1 unit to a_4 (Total 1/1 used).

Final Result: Every customer a_1, a_2, a_3, a_4 receives exactly one food item, satisfying the demand.

5. In the ($\Leftarrow$) direction of the 3-SAT $\rightarrow$ IND-SET correctness proof: given an independent set S of size $k(\varphi)$, why is it guaranteed that S cannot contain vertices for both literal x_i and literal $\neg x_i$? _____
- A Because each triangle (clause cluster) contributes at most one vertex to S , and x_i and $\neg x_i$ always appear in the same triangle.
- B Because there is a conflict edge between every x_i -vertex and every $\neg x_i$ -vertex across different triangles, so both being in S would violate independence.
- C Because the formula is in 3-CNF, so x_i and $\neg x_i$ can never appear in the same clause.
- D Because $k(\varphi)$ equals the number of variables, not the number of clauses.

Solution: B

Explanation:

Answer: B.

The construction explicitly adds a *conflict edge* between every vertex labelled x_i and every vertex labelled $\neg x_i$ in *different* triangles. Since an independent set may not contain two adjacent vertices, S can never simultaneously include both x_i and $\neg x_i$ — the conflict edge between them forbids it.

Why the other options are wrong:

- **A** is incorrect: x_i and $\neg x_i$ appear in *different* triangles (they belong to different clauses), not the same one.
- **C** is a true statement about 3-CNF but irrelevant here — it says nothing about the graph structure that prevents both from entering S .
- **D** is factually wrong: $k(\varphi)$ equals the number of *clauses*, not variables.

6. Let $\mathbf{P}$ denote the class of languages decidable in polynomial time. Language L_1 is polynomial-time reducible to language L_2 ($L_1 \leq_p L_2$). Language L_3 is polynomial-time reducible to L_2 ($L_3 \leq_p L_2$), which in turn is polynomial-time reducible to language L_4 ($L_2 \leq_p L_4$). Which of the following is/are true? _____

- A. If $L_4 \in \mathbf{P}$, then $L_2 \in \mathbf{P}$.
- B. If $(L_1 \in \mathbf{P}) \vee (L_3 \in \mathbf{P})$, then $L_2 \in \mathbf{P}$.
- C. $L_1 \in \mathbf{P} \iff L_3 \in \mathbf{P}$.
- D. If $L_4 \in \mathbf{P}$, then $(L_1 \in \mathbf{P}) \wedge (L_3 \in \mathbf{P})$.

Solution: A, D

Explanation:

For any polynomial-time reduction $A \leq_p B$, if the target language B is easily decidable ($B \in P$), then the source language A must also be easily decidable ($A \in P$).

We are given:

$$L_1 \leq_p L_2 \tag{1}$$

$$L_3 \leq_p L_2 \tag{2}$$

$$L_2 \leq_p L_4 \tag{3}$$

By transitivity of reductions, we can also deduce:

$$L_1 \leq_p L_4 \tag{4}$$

$$L_3 \leq_p L_4 \tag{5}$$

Evaluating the options:

- **A. If $L_4 \in P$, then $L_2 \in P$.**

From reduction (3), $L_2 \leq_p L_4$. If $L_4 \in P$, then L_2 must also be in P .

Result: True

- **B. If $(L_1 \in P) \vee (L_3 \in P)$, then $L_2 \in P$.**

Reductions (1) and (2) indicate L_2 is an upper bound on the complexity of L_1 and L_3 . The ease of L_1 or L_3 does not imply L_2 is easy.

Result: False

- **C. $L_1 \in P \iff L_3 \in P$.**

L_1 and L_3 share an upper bound (L_2), but have no direct relationship with each other. One can be in P independently of the other.

Result: False

- **D. If $L_4 \in P$, then $(L_1 \in P) \wedge (L_3 \in P)$.**

From deduced reductions (4) and (5), both $L_1 \leq_p L_4$ and $L_3 \leq_p L_4$. If $L_4 \in P$, any language reducible to it must be in P . Thus, both L_1 and L_3 are in P .

Result: True

Correct answer(s): A, D

7. To prove a problem X is NP-complete, which of the following is sufficient? _____

- (a) $X \leq \text{SAT}$ and $X \in \text{NP}$
- (b) $X \leq \text{SAT}$ only
- (c) $\text{SAT} \leq X$ and $X \in \text{NP}$
- (d) $\text{SAT} \leq X$ only
- (e) None of the above

Solution: C

Explanation: To prove that a problem X is NP-complete, we must show:

- $X \in \text{NP}$
 - X is NP-hard, i.e., a known NP-complete problem (such as SAT) reduces to X . This shows that every problem in NP reduces to X and since from Cook-Levin [1971], for any problem A in NP, $A \leq \text{SAT}$, $\text{SAT} \leq X$ is sufficient by transitivity of reductions.
- (a) $X \leq \text{SAT}$ and $X \in \text{NP}$
Incorrect. This shows X is no harder than SAT, but does not prove NP-hardness.
 - (b) $X \leq \text{SAT}$ only
Incorrect. Missing $X \in \text{NP}$ and wrong reduction direction.
 - (c) $\text{SAT} \leq X$ and $X \in \text{NP}$
Correct. This proves both NP-hardness and membership in NP, hence X is NP-complete.
 - (d) $\text{SAT} \leq X$ only
Incorrect. This shows NP-hardness but does not prove $X \in \text{NP}$.

8. Suppose $A \leq B$, i.e., problem A can be solved in polynomial time using a subroutine for problem B . Which of the following statements are necessarily true?_____

- (a) If $A \notin P$, then $B \notin P$
- (b) If $B \in P$, then $A \in P$
- (c) If B is NP-hard, then A is NP-hard
- (d) If A is NP-hard, then B is NP-hard

Solution: A, B, D

Explanation:

- (a) If $A \notin P$, then $B \notin P$.
True. Since $A \leq B$, if $B \in P$ then $A \in P$. Taking the contrapositive, if $A \notin P$, then $B \notin P$.
- (b) If $B \in P$, then $A \in P$.
True. Since A can be solved in polynomial time using a subroutine for B , and B itself can be solved in polynomial time, we can replace the subroutine with its polynomial-time algorithm. Hence, A can be solved in polynomial time, implying $A \in P$.
- (c) If B is NP-hard, then A is NP-hard.
False. NP-hardness does not propagate backward along reductions. B being harder does not imply A is hard; A could still be easier.
- (d) If A is NP-hard, then B is NP-hard.
True. Since A is NP-hard, every problem in NP reduces to A . Given $A \leq B$, by transitivity of reductions, every problem in NP reduces to B . Hence, B is NP-hard.

Final Answer: (a), (b) and (d) are correct.

9. Consider a directed graph $G = (V, E)$ with a source s , a sink t , and positive integer capacities $c(u, v)$ for all $(u, v) \in E$. Let f be a flow in the network and (S, T) be an $s - t$ cut.

Which of the following statements are correct? _____

- A. If the capacity of every edge in the network is increased by exactly 1 unit, the original minimum cut (S, T) will remain a minimum cut in the new network.
- B. In any flow network, there exists a unique minimum $s - t$ cut if and only if the maximum flow value is strictly less than the sum of all edge capacities.
- C. If f is a maximum flow, then for every minimum cut (S, T) , there is no flow on any edge (u, v) such that $u \in T$ and $v \in S$.
- D. Suppose all edge capacities are distinct. Then there is a unique minimum $s - t$ cut in the network.
- E. If we multiply all edge capacities by a positive constant $k > 0$, the set of all minimum cuts in the network remains unchanged.

Solution: C, E

Explanation:

Statement A: False If every edge capacity is increased by $\epsilon = 1$, the new capacity is $c'(u, v) = c(u, v) + 1$. The new capacity of a cut (S, T) becomes:

$$C'(S, T) = \sum_{u \in S, v \in T} (c(u, v) + 1) = C(S, T) + |E_{S, T}|$$

where $|E_{S, T}|$ is the number of edges crossing the cut. Since the increase depends on the *count* of edges in the cut, a cut with fewer edges might become the new minimum even if it wasn't originally.

Statement B: False The uniqueness of a minimum cut is a structural property. $f_{\max} < \sum c(u, v)$ simply means the network is not fully saturated. A unique min-cut can exist even if $f_{\max} = \sum c(u, v)$ (e.g., a single-edge graph).

Statement C: True For any flow f and any cut (S, T) , the net flow is:

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in T, v \in S} f(u, v)$$

The Max-Flow Min-Cut Theorem states $|f| \leq C(S, T)$. Equality holds if and only if:

- (a) $f(u, v) = c(u, v)$ for all $u \in S, v \in T$ (Forward edges are saturated).
- (b) $f(u, v) = 0$ for all $u \in T, v \in S$ (Backward edges have zero flow).

Statement D: False Distinct edge capacities do not guarantee a unique min-cut. For example, if $c(e_1) = 2, c(e_2) = 3$, and $c(e_3) = 5$, then the cut $\{e_1, e_2\}$ and the cut $\{e_3\}$ both have capacity 5, even though all individual capacities are distinct.

Statement E: True Let $k > 0$. If $c'(u, v) = k \cdot c(u, v)$, then for any cut (S, T) :

$$C'(S, T) = \sum_{u \in S, v \in T} k \cdot c(u, v) = k \sum_{u \in S, v \in T} c(u, v) = k \cdot C(S, T)$$

Since k is a positive constant, the relative ordering of all cut capacities remains the same. If $C(S_1, T_1)$ was minimal, $k \cdot C(S_1, T_1)$ remains minimal.

10. Which of the following statements is **NOT true** regarding the Edmonds–Karp algorithm? Assume all edge capacities to be integers. _____

- a) In each augmentation (sending flow along an augmenting path from S to T), at least one edge on the chosen shortest path (in terms of number of edges, considering current residual capacities) becomes saturated (i.e., its residual capacity becomes zero).
- b) The shortest path distance (number of edges from S to T) in the residual graph never decreases over successive augmentations.
- c) Edmonds–Karp is considered stable because the shortest path distance can only stay the same or increase over time.
- d) If BFS is replaced by DFS (i.e., we pick any augmenting path instead of the shortest), the maximum flow value computed may differ depending on the sequence of augmenting paths chosen.

Solution: D

Explanation:

We analyze each statement regarding the Edmonds–Karp algorithm:

(a) True

In each augmentation, the flow pushed is equal to the bottleneck capacity:

$$c_f(P) = \min\{c_f(u, v) : (u, v) \in P\}$$

At least one edge on the chosen shortest path will have residual capacity equal to this minimum and becomes:

$$c_f(u, v) - c_f(P) = 0$$

Hence, at least one edge becomes saturated.

(b) True

A key property of Edmonds–Karp is that the shortest path distance (the number of edges in between the source and the vertex) from source S to any vertex v in the residual graph is monotonically non-decreasing:

$$\delta_{f'}(S, v) \geq \delta_f(S, v)$$

Thus, the shortest path from S to T never decreases over time.

(c) True

This is a direct consequence of part (b). Since distances never decrease, the algorithm progresses in a stable manner toward termination.

(d) False

The maximum flow value is independent of the choice of augmenting paths. This follows from the Max-Flow Min-Cut Theorem.

Even if BFS is replaced by DFS (i.e., using the general Ford–Fulkerson method):

- The sequence of augmenting paths may differ

- The runtime may change
- But the final maximum flow value remains the same (if the algorithm terminates)

Thus, the statement claiming that the maximum flow may change is incorrect.

Final Answer: D